GPU Architecture

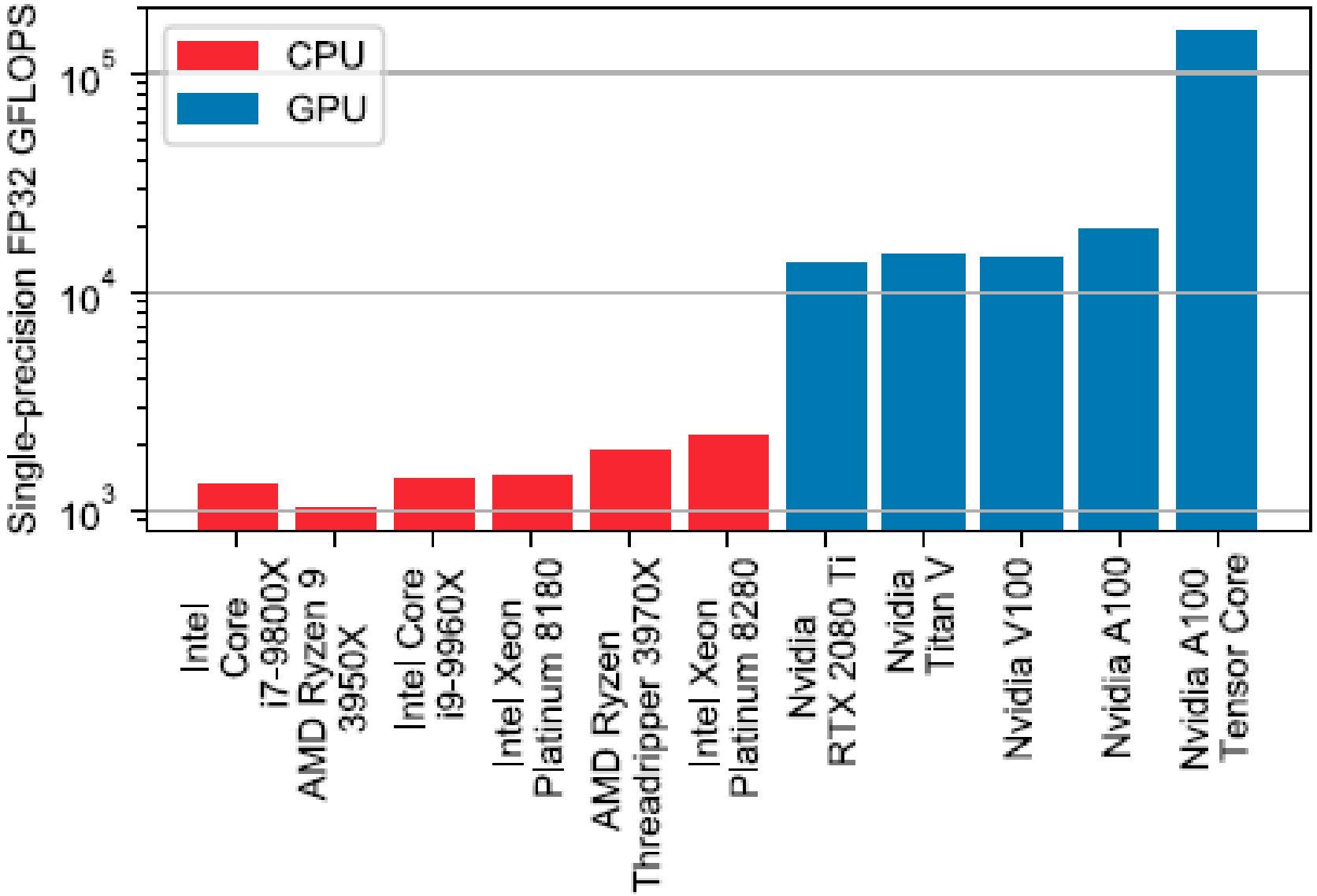
Sparsh Mittal

MOTIVATION FOR STUDYING GPU

Evaluation Metrics

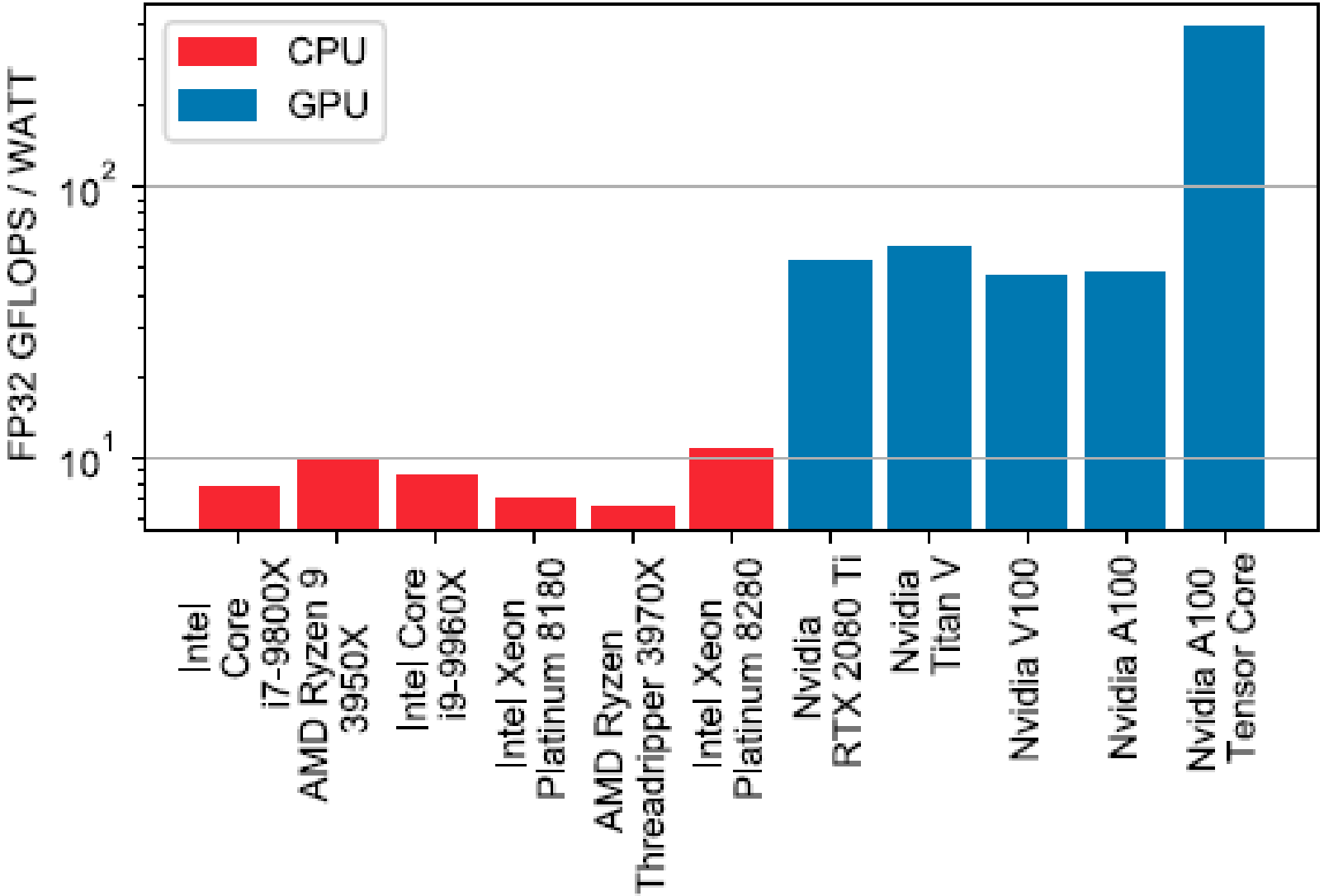
- We will show
  - Peak-throughput in terms of Giga FLOPS (floating-point operations per second)
  - Energy efficiency in terms of GFLOPS/watt, which shows the throughput achieved for each watt of power consumed

Performance Comparison



Capra, M., et al. "Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead." IEEE Access 8 (2020): 225134-225180.

Performance/Watt Comparison



Capra, M., et al. "Hardware and software optimizations for accelerating deep neural networks: Survey of current trends, challenges, and the road ahead." IEEE Access 8 (2020): 225134-225180.

FIGURE 2. Transistor density scaling of production GPUs launched in the last five years. GPU lifetime in the market is typically three years after launch.

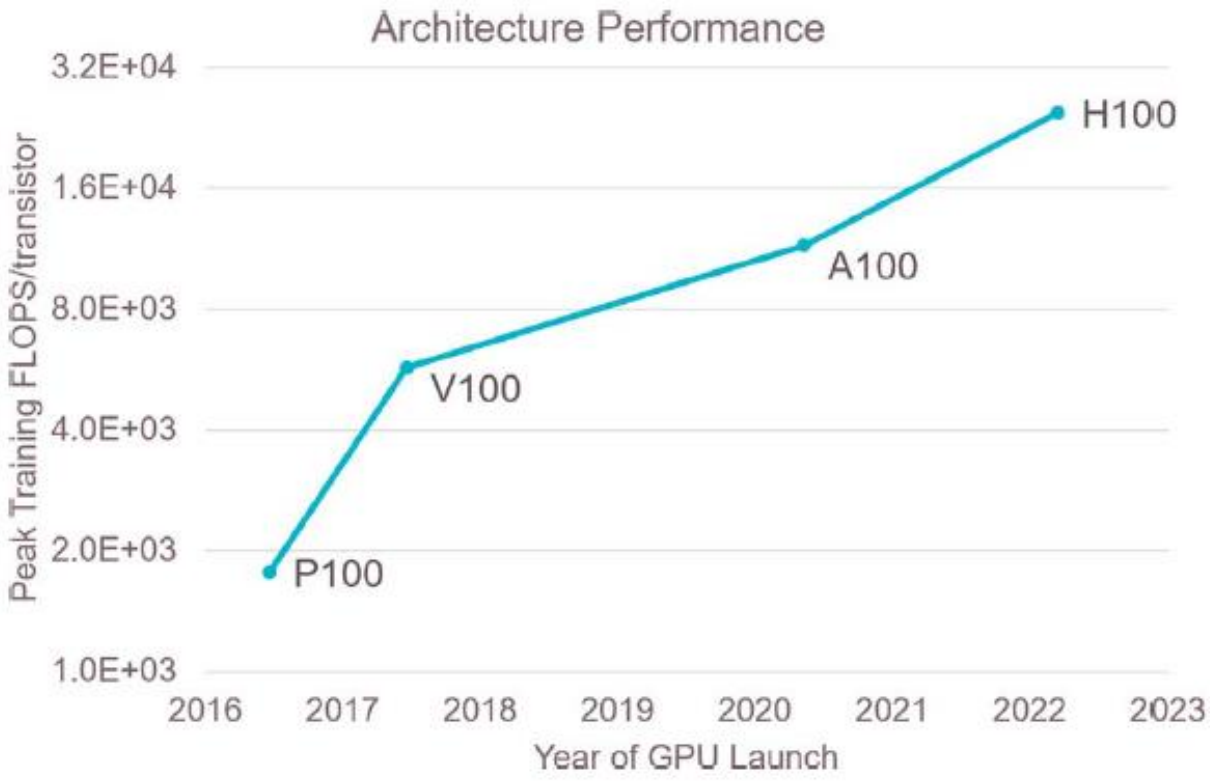
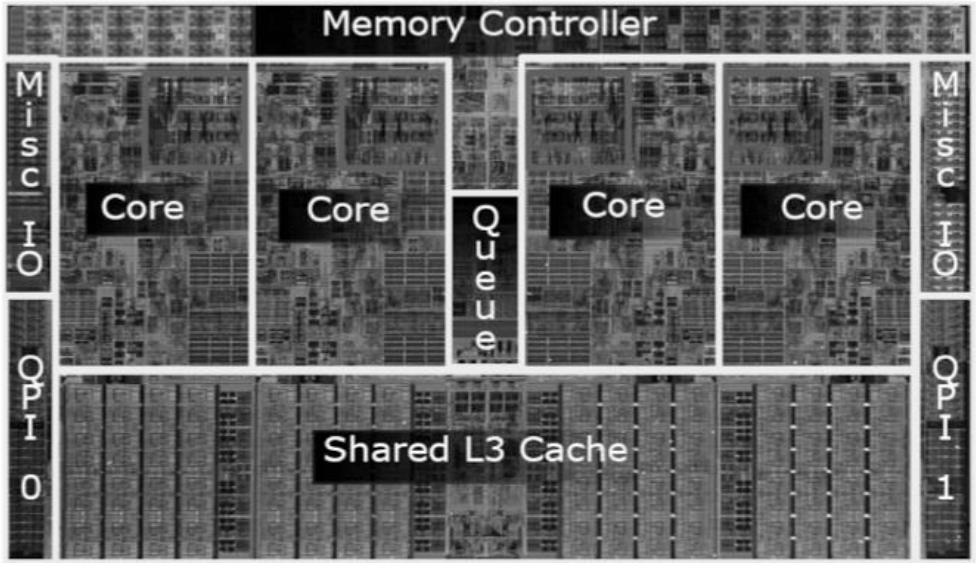
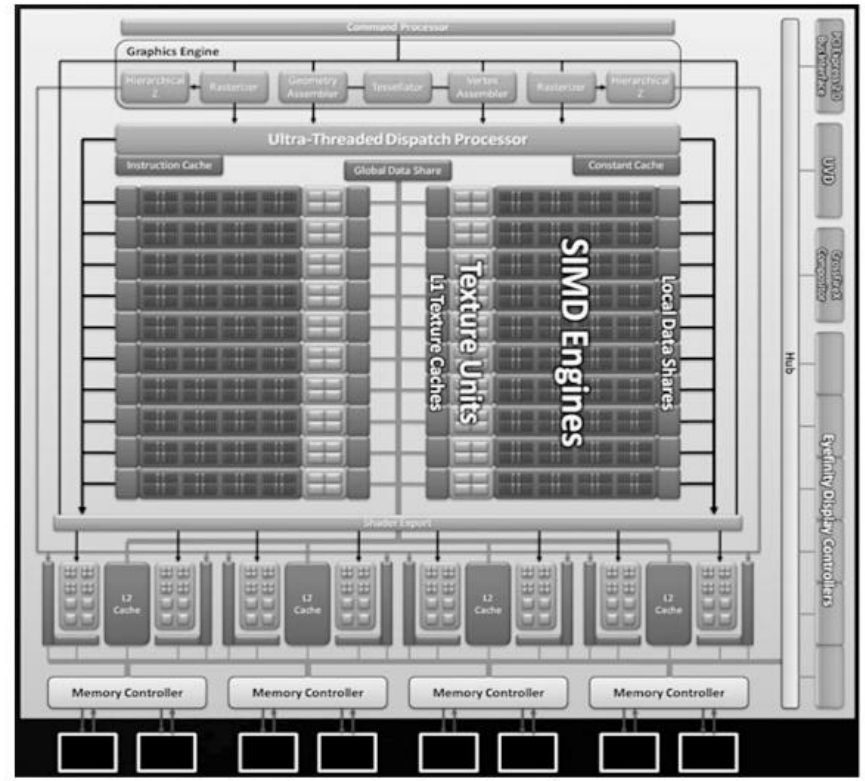


FIGURE 3. Architecture performance scaling of production GPUs in the last five years. GPU lifetime in the market is typically three years after launch.

Intel Nehalem 4-core

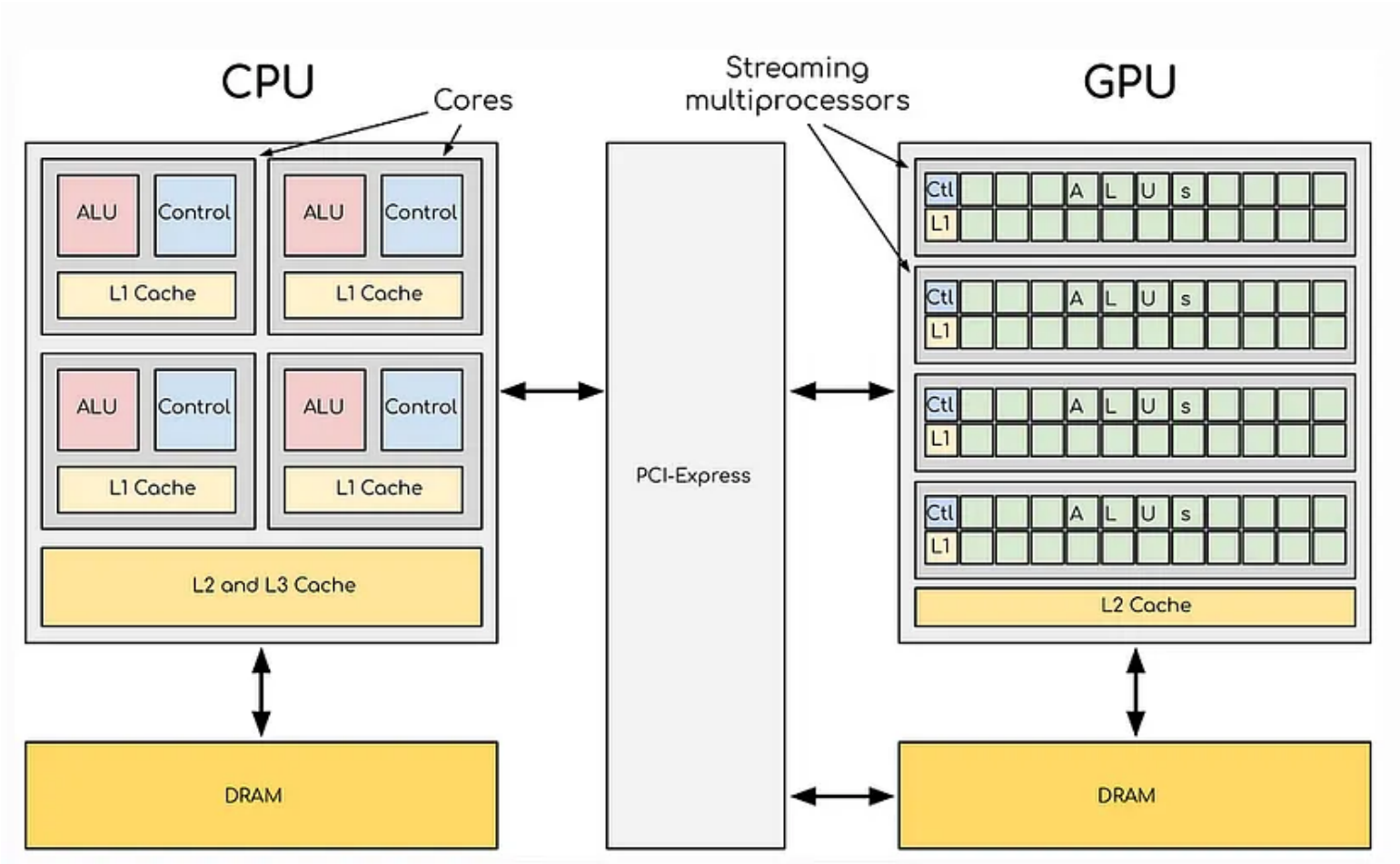


AMD Radeon 5870



130W, 263mm^2
32 GB/s BW, **106 GFLOPs** (SP)
Big caches (8MB)
Out-of-order
0.8 GFLOPs/W

188W, 334mm^2
154 GB/s BW, **2720 GFLOPs** (SP)
Small caches (<1MB)
Hardware thread scheduling
14.5 GFLOPs/W

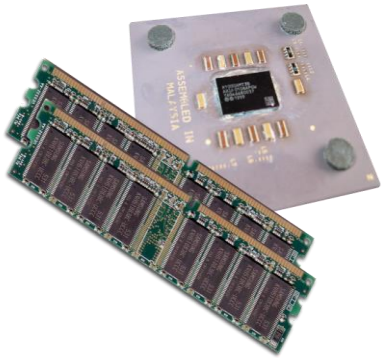

CPU
Cores
ALU
Control
L1 Cache
L2 and L3 Cache
DRAM
PCI-Express
Streaming multiprocessors
GPU
Ctl
L1
A L U s
L2 Cache
DRAM

HETEROGENEOUS COMPUTING

Images are courtesy of: https://www.nvidia.com/docs/IO/116711/sc11-cuda-c-basics.pdf

Heterogeneous Computing

- Terminology:
  - *Host:* the CPU and its memory (host memory)
  - *Device:* the GPU and its memory (device memory)



Host

Device

Heterogeneous Computing

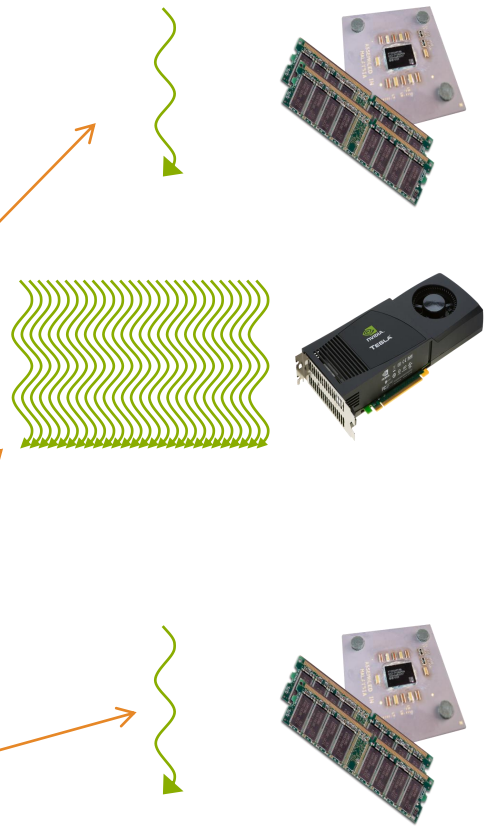
```
#include <iostream>
#include <algorithm>

using namespace std;

#define N          1024
#define RADIUS     3
#define BLOCK_SIZE 16

__global__ void stencil_1d(int *in, int *out) {
        __shared__ int temp[BLOCK_SIZE + 2 * RADIUS];
        int gindex = threadIdx.x + blockIdx.x * blockDim.x;
        int lindex = threadIdx.x + RADIUS;

        // Read input elements into shared memory
        temp[lindex] = in[gindex];
        if (threadIdx.x < RADIUS) {
                temp[lindex - RADIUS] = in[gindex - RADIUS];
                temp[lindex + BLOCK_SIZE] = in[gindex + BLOCK_SIZE];
        }

        // Synchronize (ensure all the data is available)
        __syncthreads();

        // Apply the stencil
        int result = 0;
        for (int offset = -RADIUS ; offset <= RADIUS ; offset++)
                result += temp[lindex + offset];

        // Store the result
        out[gindex] = result;
}

void fill_ints(int *x, int n) {
        fill_n(x, n, 1);
}

int main(void) {
        int *in, *out;          // host copies of a, b, c
        int *d_in, *d_out;      // device copies of a, b, c
        int size = (N + 2*RADIUS) * sizeof(int);

        // Alloc space for host copies and setup values
        in  = (int *)malloc(size); fill_ints(in,  N + 2*RADIUS);
        out = (int *)malloc(size); fill_ints(out, N + 2*RADIUS);

        // Alloc space for device copies
        cudaMalloc((void **)&d_in,  size);
        cudaMalloc((void **)&d_out, size);

        // Copy to device
        cudaMemcpy(d_in,  in,  size, cudaMemcpyHostToDevice);
        cudaMemcpy(d_out, out, size, cudaMemcpyHostToDevice);

        // Launch stencil_1d() kernel on GPU
        stencil_1d<<<N/BLOCK_SIZE,BLOCK_SIZE>>>(d_in + RADIUS,
d_out + RADIUS);

        // Copy result back to host
        cudaMemcpy(out, d_out, size, cudaMemcpyDeviceToHost);

        // Cleanup
        free(in); free(out);
        cudaFree(d_in); cudaFree(d_out);
        return 0;
}
```

Parallel function

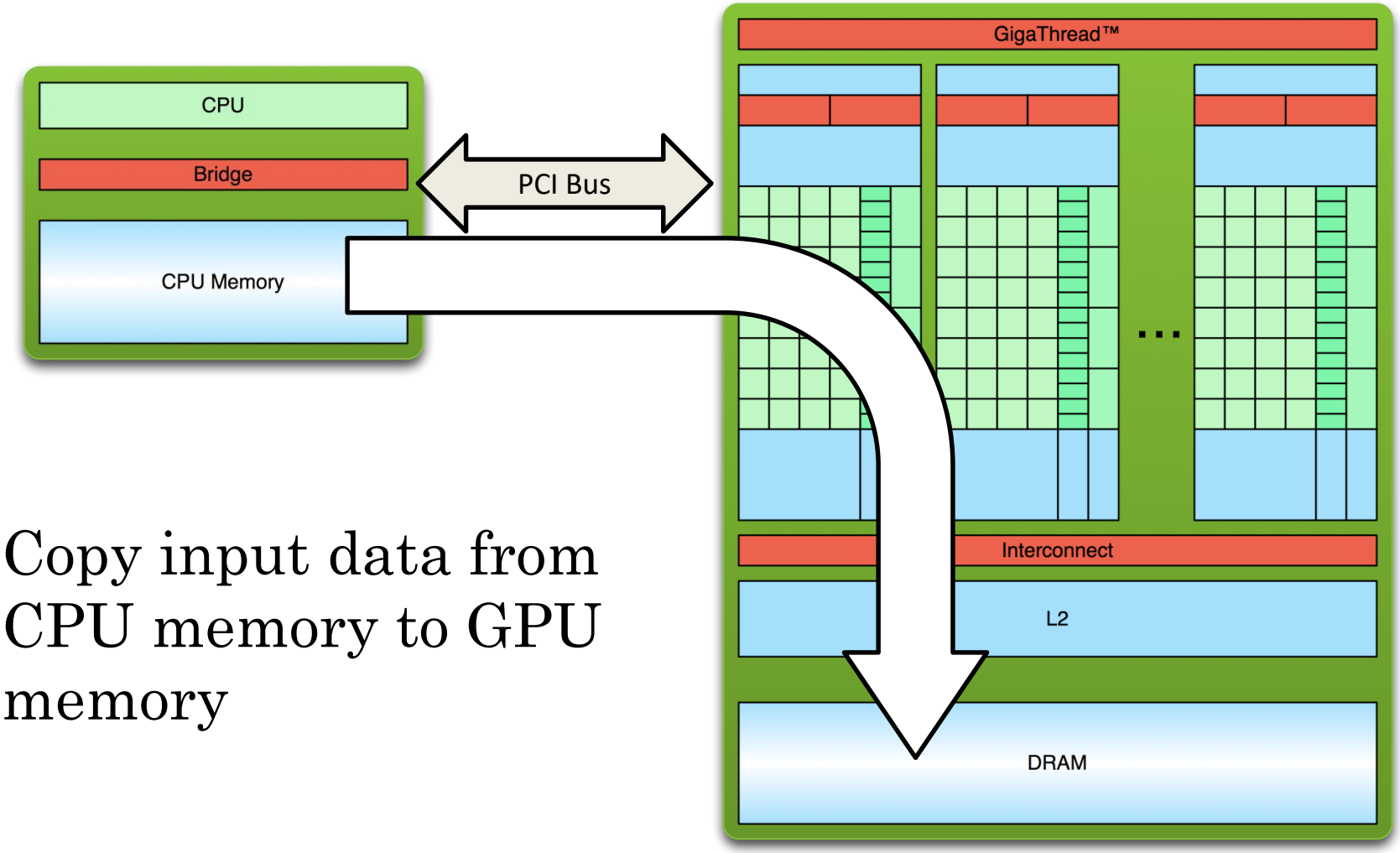
Serial code

Parallel code

Serial code



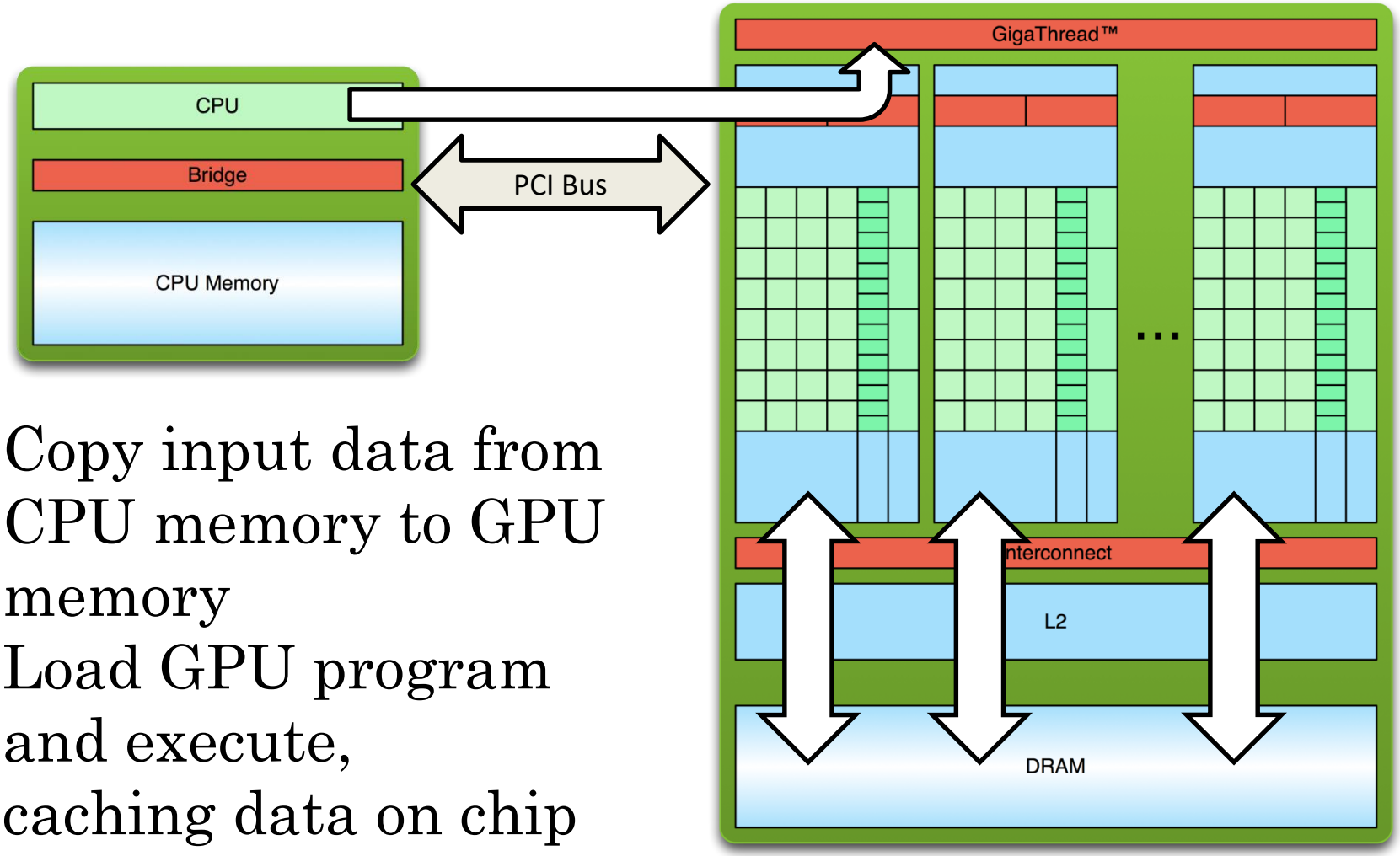
Processing Flow in Heterogeneous Computing



1. Copy input data from CPU memory to GPU memory

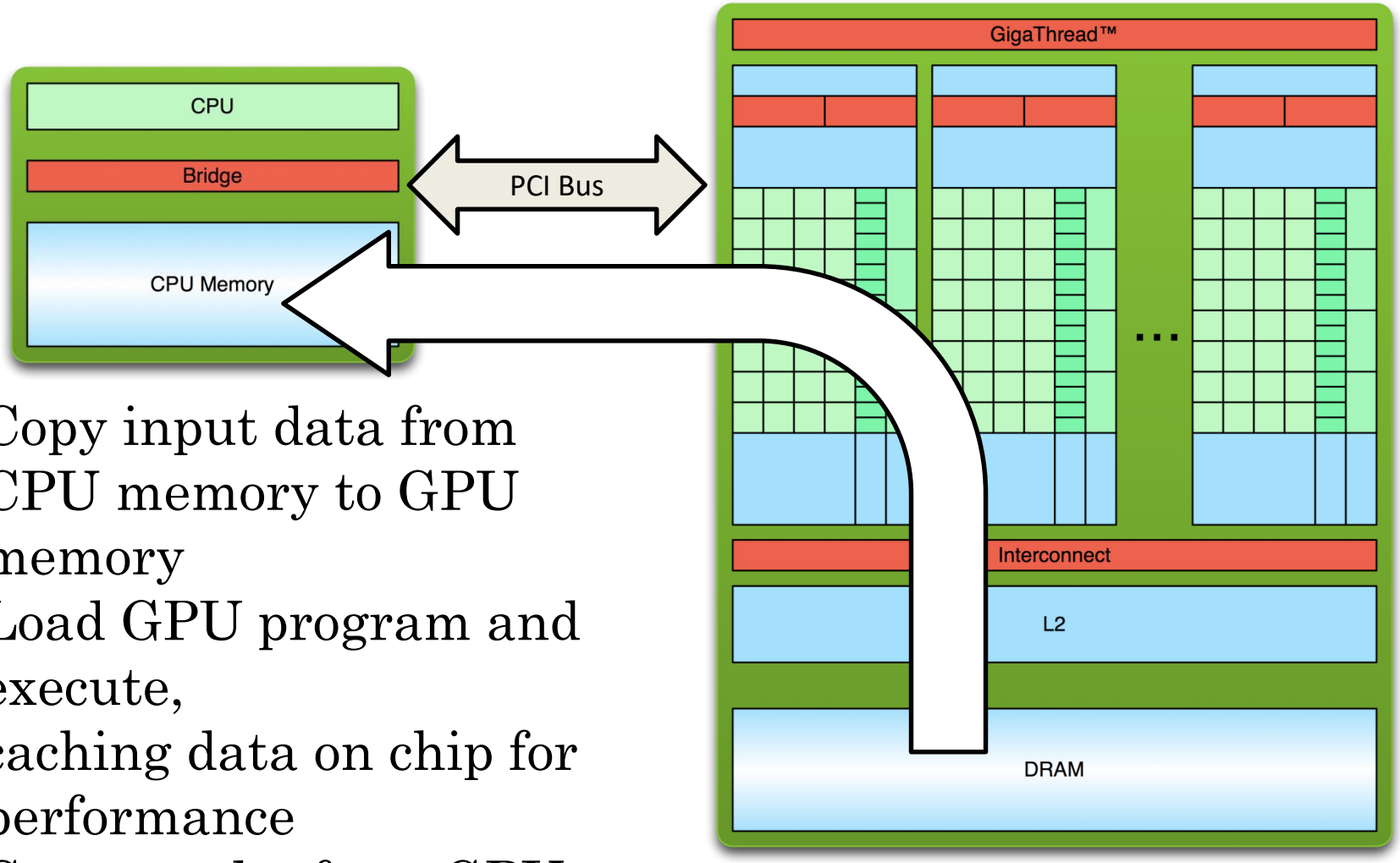
PCI Bus = Peripheral component interconnect bus

Processing Flow in Heterogeneous Computing



1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute, caching data on chip for performance

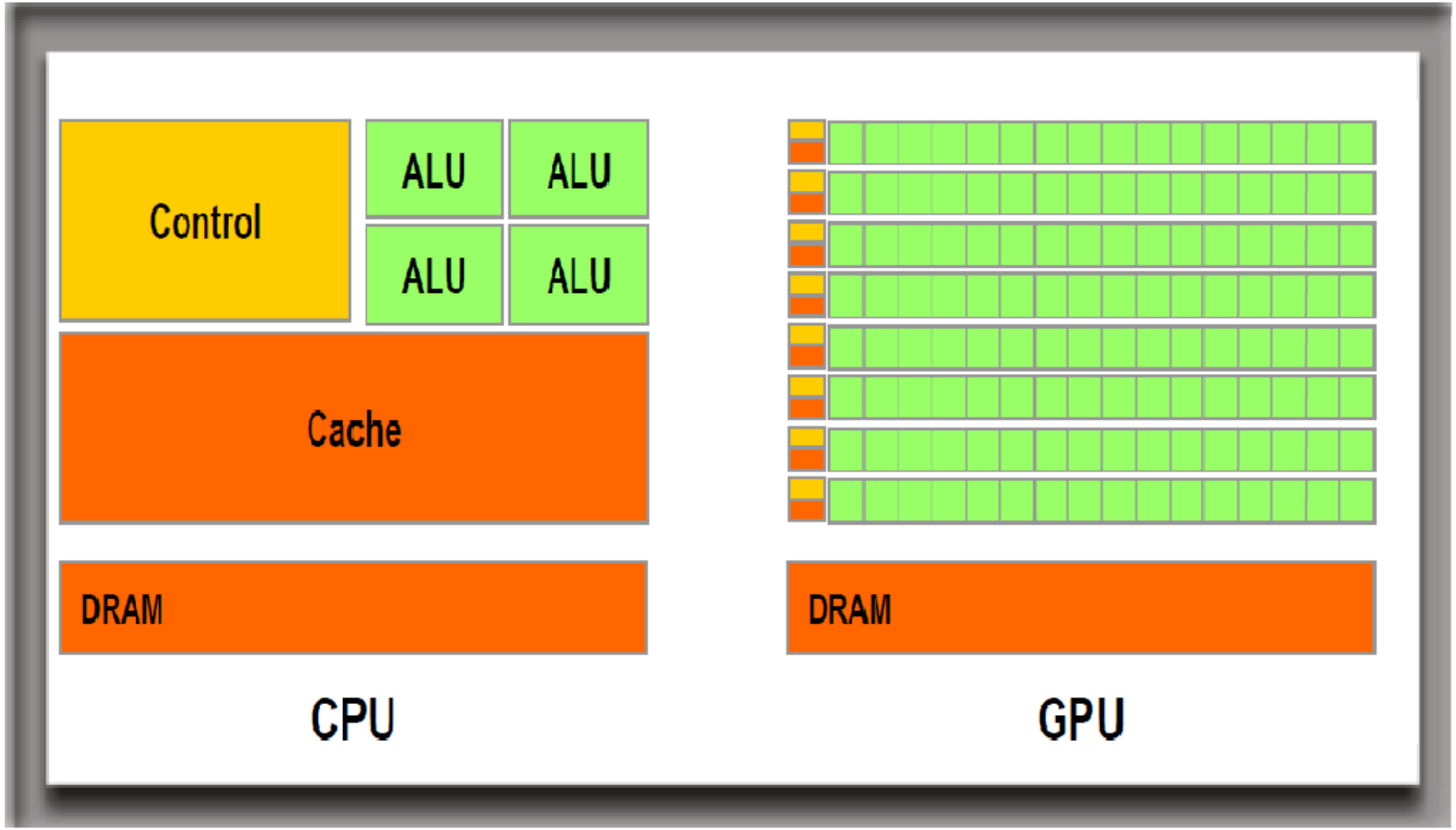
Processing Flow in Heterogeneous computing



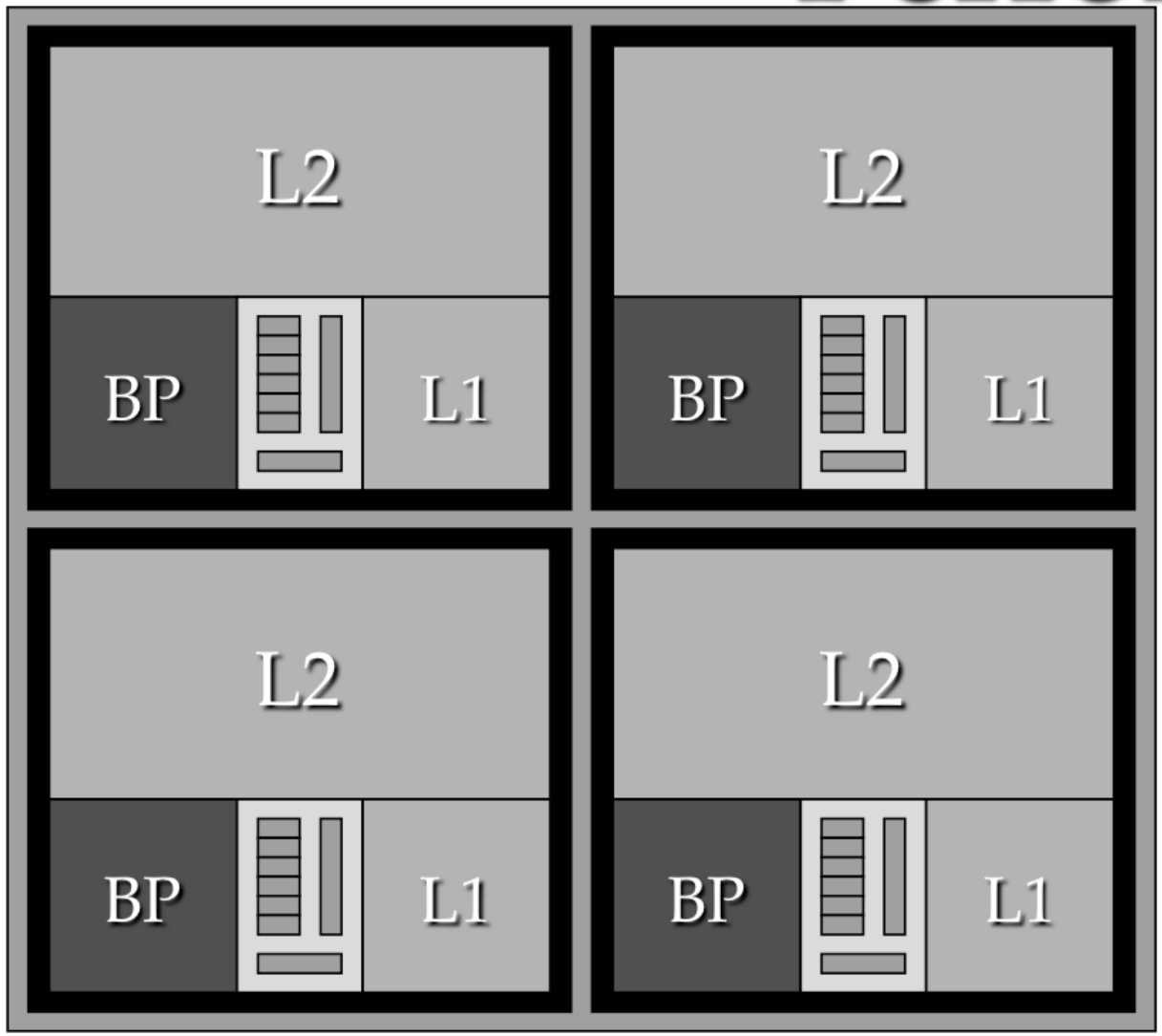
1. Copy input data from CPU memory to GPU memory
2. Load GPU program and execute,
   caching data on chip for performance
3. Copy results from GPU memory to CPU memory

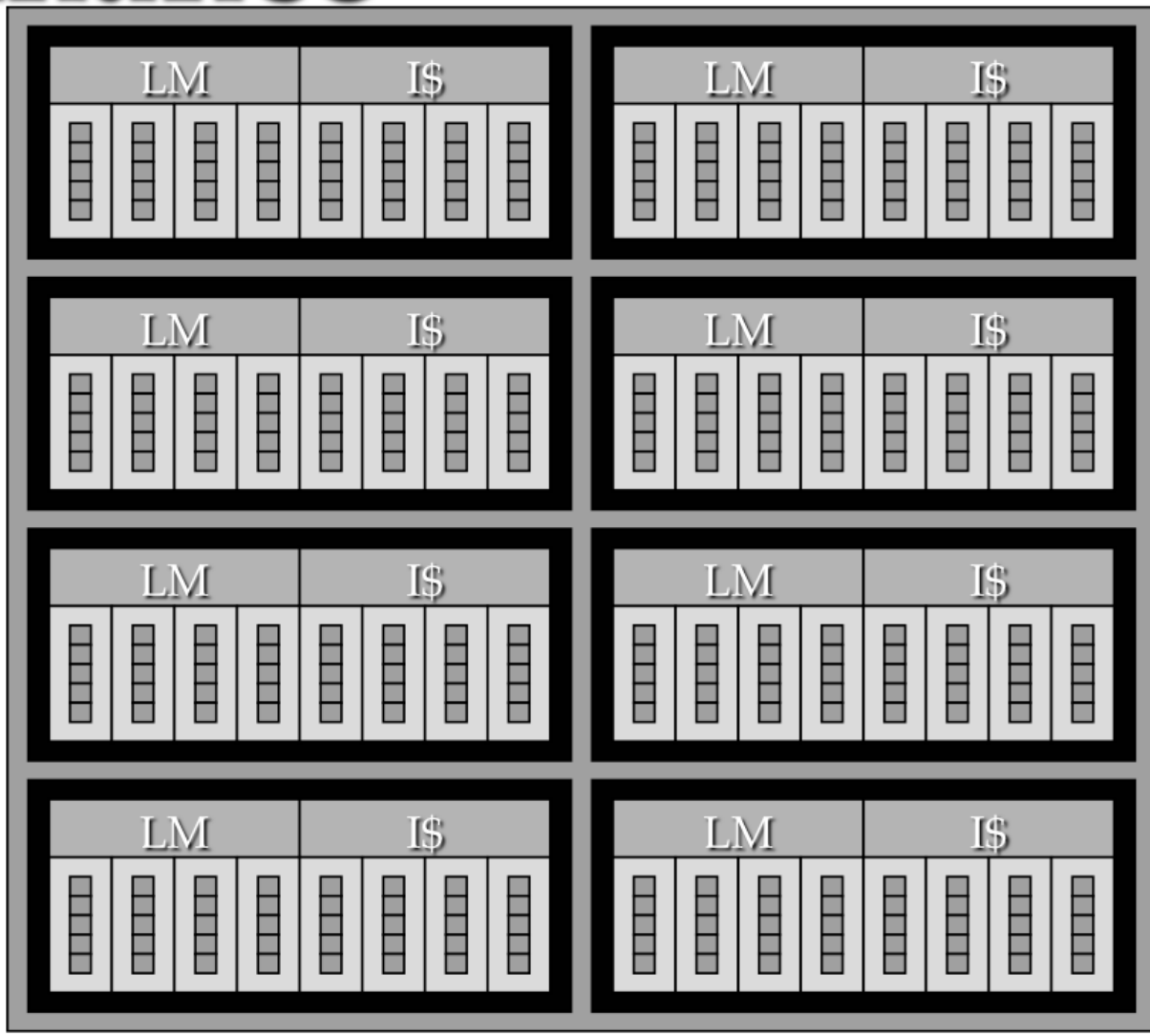
Architectural Differences Between CPUs and GPUs

GPU vs. CPU



ALU= Arithmetic logic unit DRAM= Dynamic random access memory





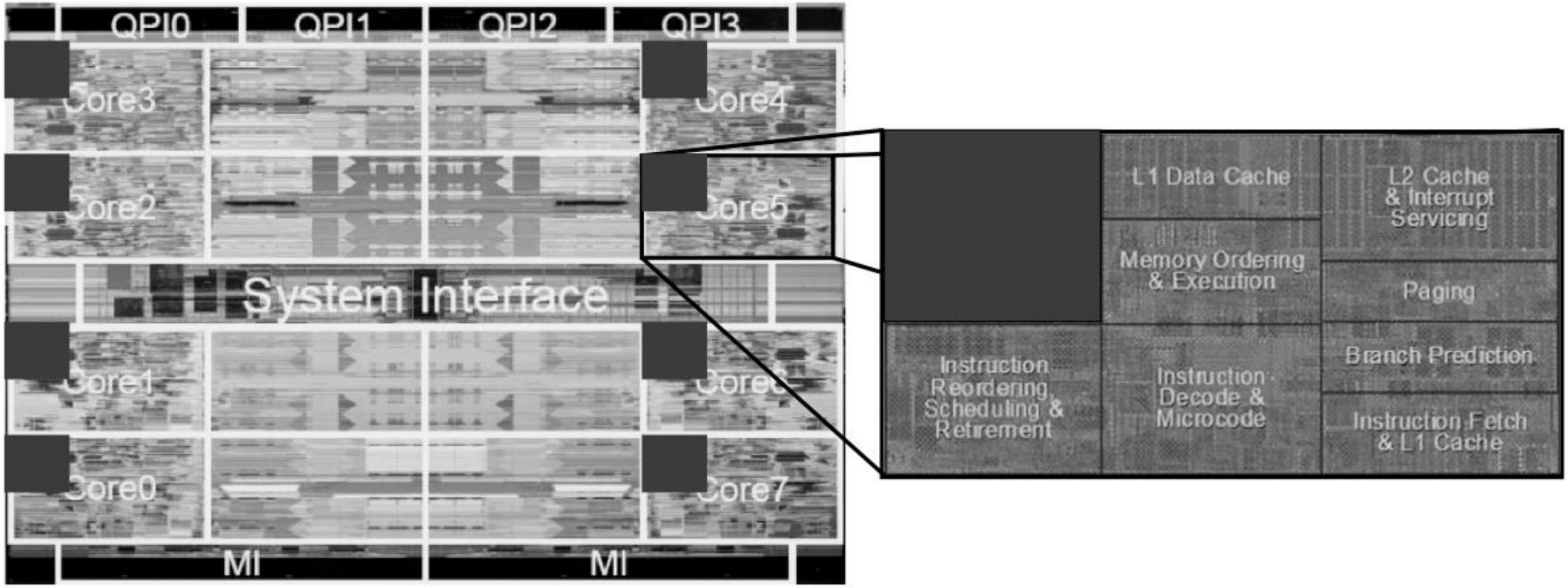
4 Massive CPU Cores: Big caches, branch predictors, out-of-order, multiple-issue, speculative execution, double-precision…
About 2 IPC per core, 8 IPC total @3GHz

8*8 Wimpy GPU Cores: No caches, in-order, single-issue, single-precision…

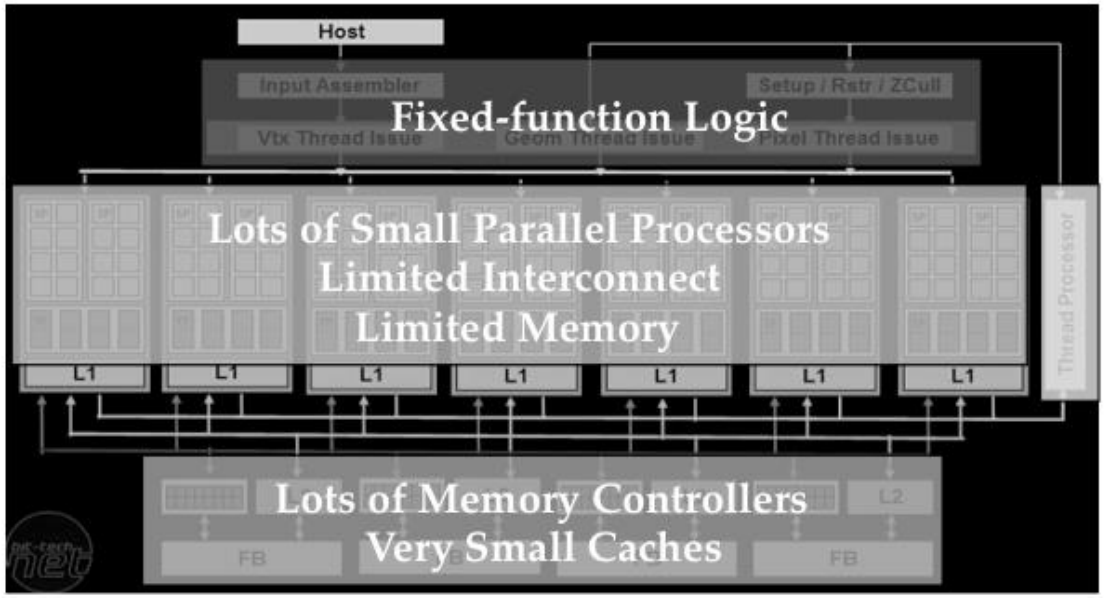
About 1 IPC per core, 64 IPC total @1.5GHz

Example CPU: Intel 8-core Nehalem (45nm)

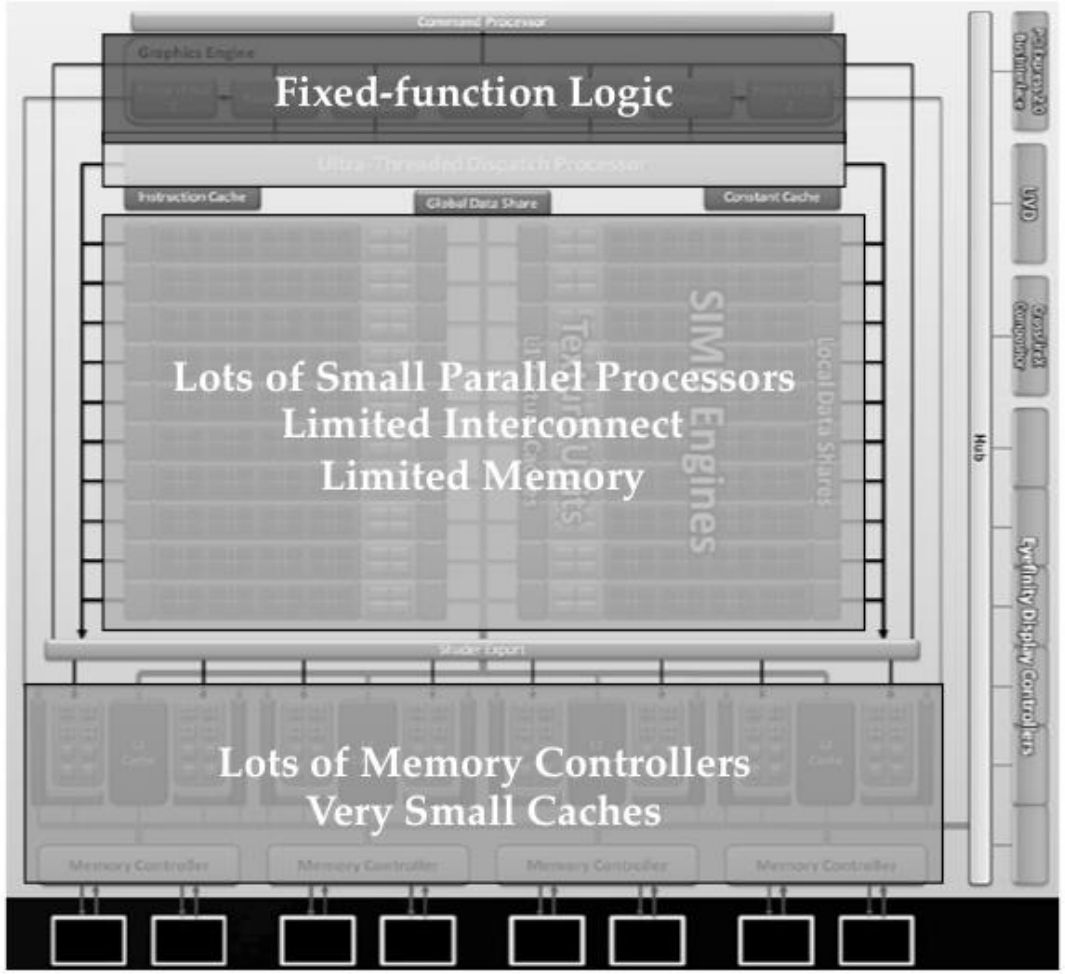
- Intel's high-end server chip uses a very small amount of the area to actually compute. Most area is cache!



Example GPUs



NVIDIA G80



AMD 5870 GPU

The Tradeoff

Optimizes
LATENCY

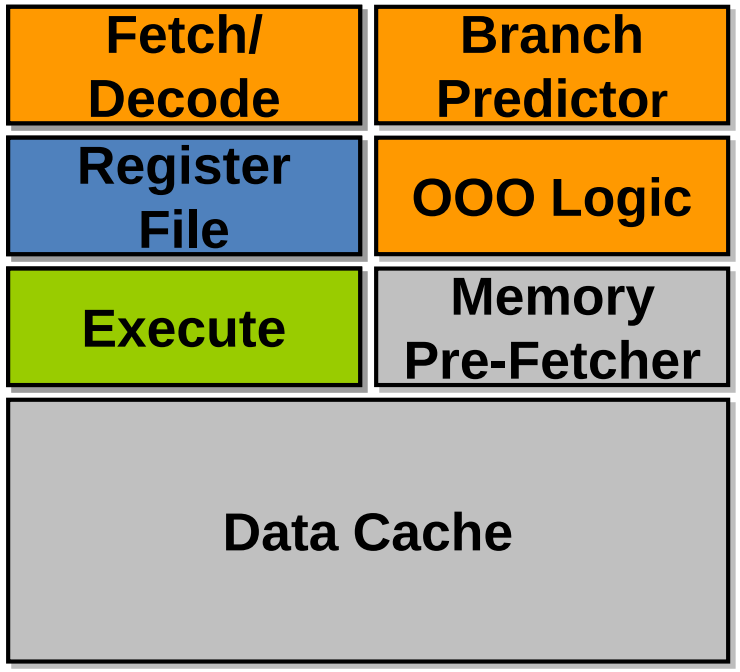
CPU

Optimizes
THROUGHPUT

GPU

CPU vs. GPU: Architectural Difference 1

CPU



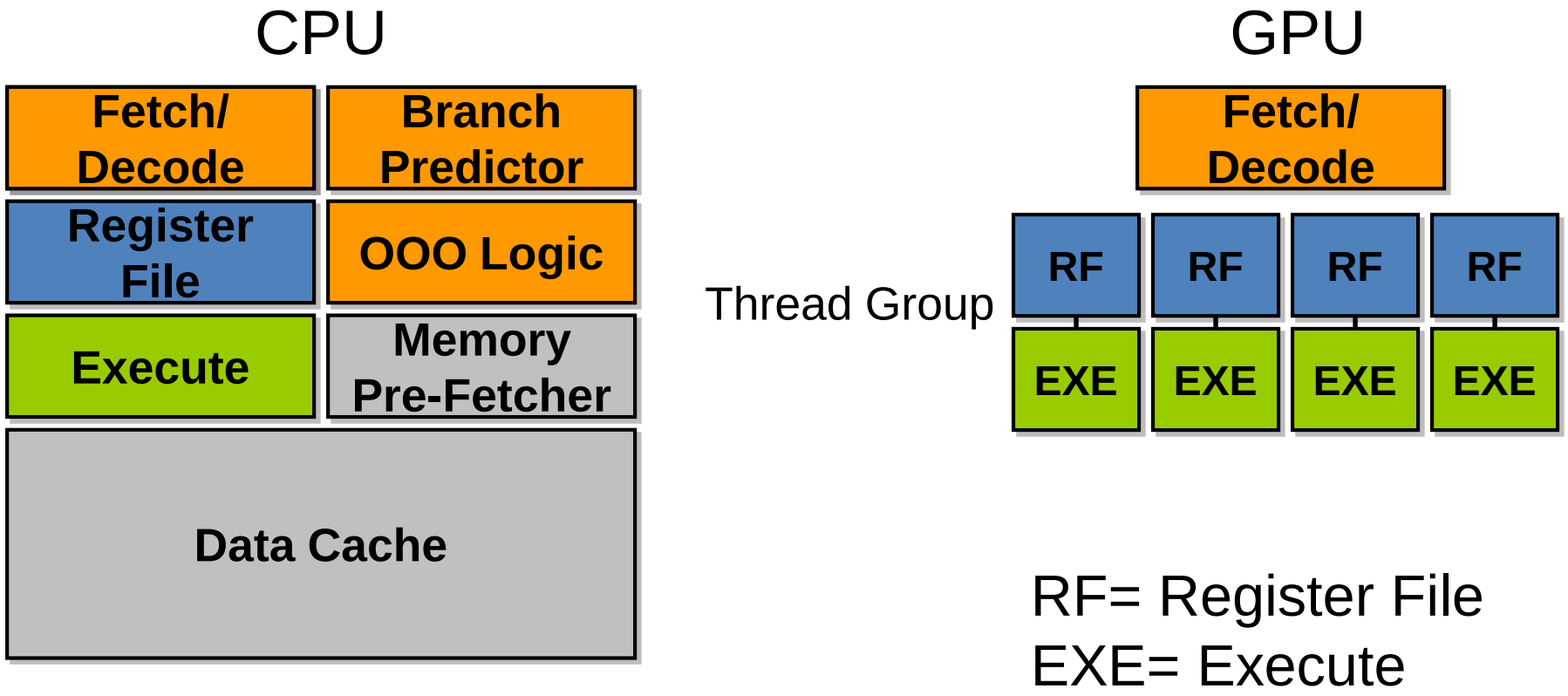
GPU

Avoid structures that only improve single-thread performance

OOO = out-of-order

Slide courtesy: Michael Boyer

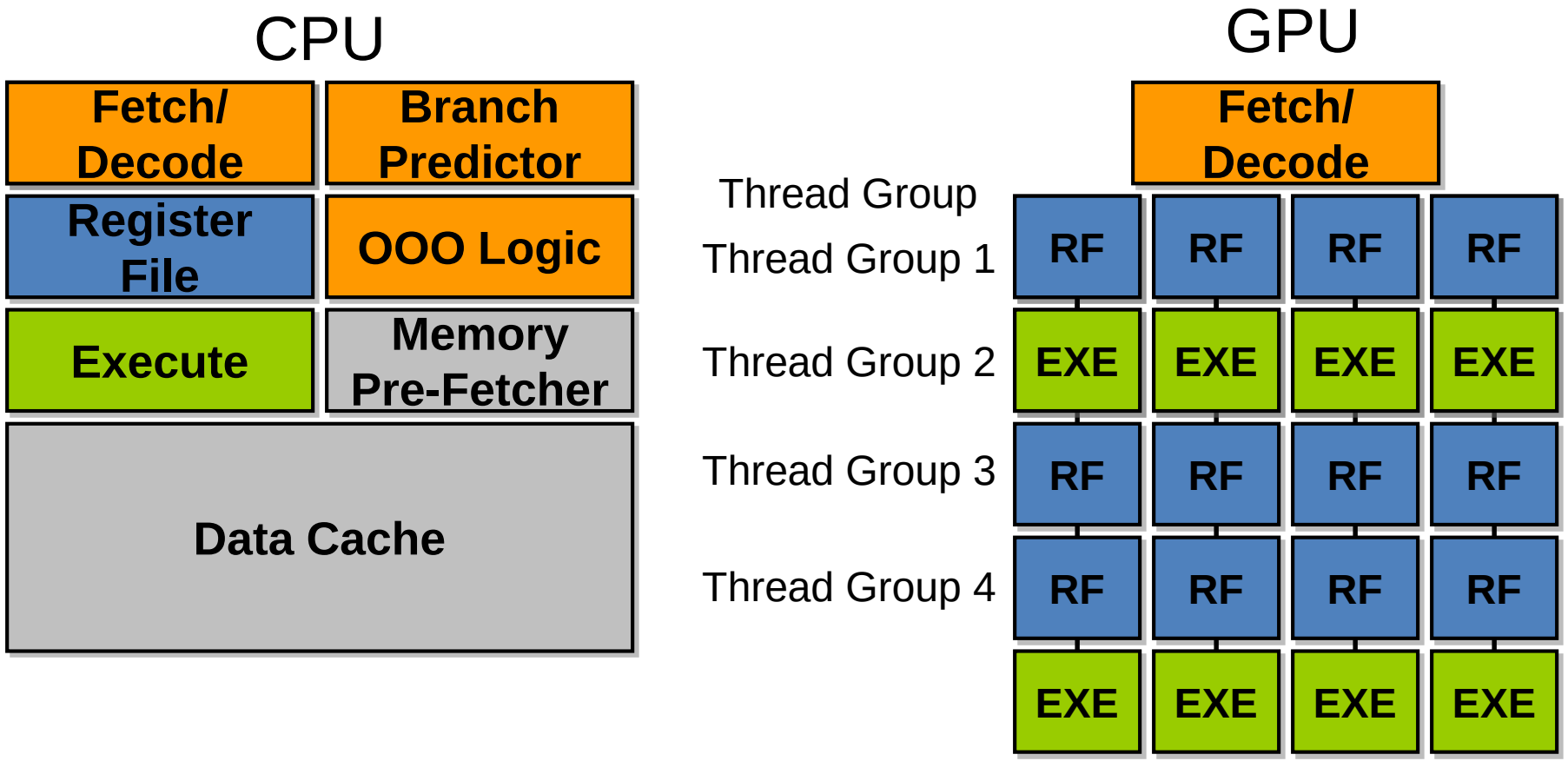
CPU vs. GPU: Architectural Difference 2



SIMD processing: amortize the overhead of control logic across multiple execution units (parallelism in space)

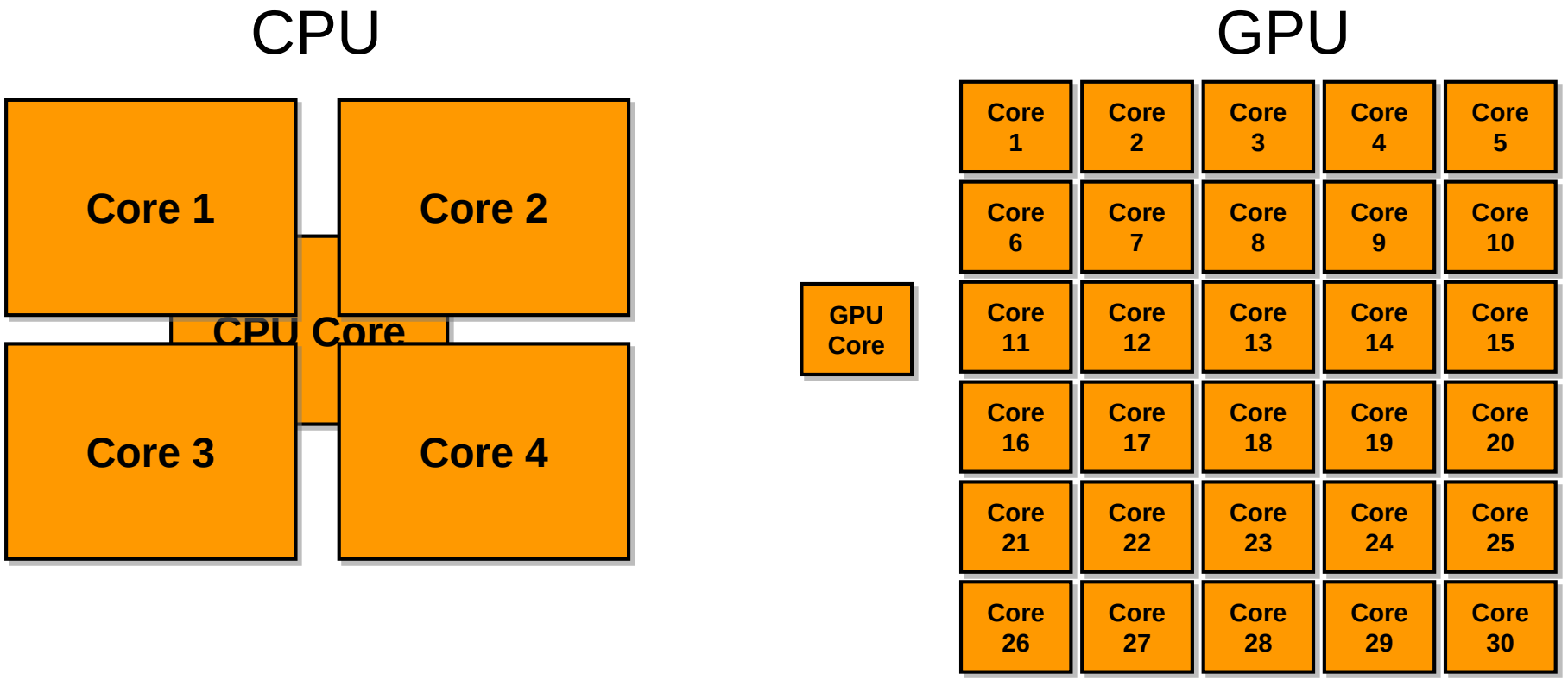
SIMD = single instruction multiple data

CPU vs. GPU: Architectural Difference 3



Use multiple groups of threads to keep execution units busy and hide memory latency (parallelism in time)

CPU vs. GPU: Architectural Difference 4



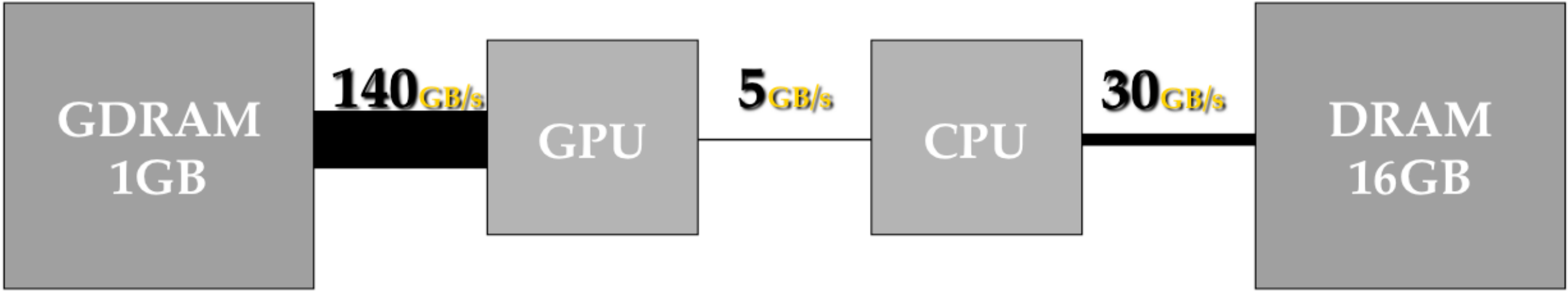
Replicate cores to leverage more parallelism (parallelism in space)

Further comments

- **Differences between GPU and CPU threads**
- § GPU threads are extremely lightweight
- – Very little creation overhead
- § GPU needs 1000s of threads for full efficiency
- – Multi-core CPU needs only a few

Component	CPU	GPU
Core Count	Few but powerful (4-64 cores)	Thousands of smaller cores (3,000-15,000+)
Clock Speed	High (3-5 GHz)	Lower (1-2 GHz)
Cache Memory	Large (L1, L2, L3 caches)	Smaller but optimized for throughput
Control Units	Complex branch prediction, out-of-order execution	Simpler control logic, optimized for throughput
ALUs	Few complex ALUs	Thousands of simpler ALUs
Memory Interface	Optimized for low latency	Optimized for high bandwidth
Thread Management	Dozens to hundreds of threads	Thousands to millions of threads

Limitations of GPUs



- Getting data in and out of GPUs is slow.
- if you have to move data to/from the CPU frequently (or at all, really) you will see very little performance improvement

- The latest NVIDIA H100 GPUs have 3.43× more compute and 1.75× more power compared to their predecessor A100 GPUs. However, their memory bandwidth only grew by 1.6×, with no increase in memory capacity.

	A100	H100	Ratio
TFLOPs	19.5	66.9	3.43×
HBM capacity	80GB	80GB	1.00×
HBM bandwidth	2039GBps	3352GBps	1.64×
Power	400W	700W	1.75×
NVLink	50Gbps	100Gbps	2.00×
Infiniband	200GBps	400GBps	2.00×
Cost per machine [5]	$17.6/hr	$38/hr	2.16×

References

- https://medium.com/@rohithreddy66666/decoding-cpu-vs-gpu-a-detailed-exploration-of-nvidia-and-amd-gpu-architectures-e09ebfb594ed